# RECITATION

**Working With Linux Environment**

What is Linux?

Open-source, Unix-like operating system. Powers servers, phones, embedded devices.

Why Linux?

- Free, stable, secure
- Industry standard for servers, dev environments
- Essential for developers, sysadmins, and researchers

# RECITATION

**Basic Commands**

`whoami` → shows your current username

`echo "Hello"` → prints text to screen

`which $SHELL` → path of your current shell (e.g., /bin/bash)

`cat file.txt` → display file contents

`head -n 5 file.txt` → show first 5 lines

`tail -n 5 file.txt` → show last 5 lines

`ls -l` → list files with details

vim <filename> / nano <filename>

# RECITATION

**Flags**

Enable you to expand functionality

`ls -l` → (long format)

`ls -a` → (show hidden)

`ls -la` → (both)

# RECITATION

**Shell Script**

Script = file containing shell commands executed sequentially

Save with .sh extension. shebang*

Run with bash script.sh or ./script.sh (needs chmod +x) **

* shebang specifies which shell to use

** File permissions

# RECITATION

**Variables**

Assign with `VAR=value` (no spaces)

Access with `$VAR`

Temporary: exists only in current shell/session

Any program or subshell you start does not see it.

`export VAR=value`

Sets a shell variable and marks it as an environment variable.

It becomes part of the shell's environment and is inherited by all child processes.

# RECITATION

**User Interaction**

Use read to accept input

```
echo "Enter your name:"
read NAME
echo "Hi $NAME"
```

# RECITATION

**Arguments & Positional Arguments**

```
echo Hello World
0    1     2
```

Pass args to script: ./script.sh arg1 arg2

Access as $1, $2, ... $@

# RECITATION

**Piping**

Sending the output of a command as input to another command

cat Hello.txt **|** grep world

# RECITATION

**Redirection: Output**

As the name implies

> - Write to file

>> - Append to file

# RECITATION

**Redirection: Input**

Read from file

```
wc -w hello.txt -> wc -w < hello.txt
```

<< - used for multiple lines

# RECITATION

**Conditionals - if/elif/else**

Condition (<, >, ==, != ):

${1, , } = John

```
if [ <condition> ] ; then
    echo Hello world! I’m a DSA Expert
fi
```

# RECITATION

**Conditionals - Case**

Instead of multiple if else, you can have a switch case

```
case <condition> in
    <check1> ) { do something } ;;
    <check2> ) { do something else } ;;
    * <default> )  ::
esac
```

# RECITATION

**Arrays**

Holds multiple objects

```
MY_ARRAY = (one two three four)
```

//each separated by a space is an object not a necessarily a string

Read an array:

```
echo ${MY_ARRAY} *
echo ${MY_ARRAY[@]} **
echo ${MY_ARRAY[0]}
```

//outputs only the first element

//outputs all elements

//outputs a specific element

# RECITATION

**For Loop**

Reiteration without manual control

```
for item in ${MY_ARRAY[@]}; do
    ##do something with $item
done
```

# RECITATION

**Functions**

```
functionName() { #do something ridiculous }
```

function call:

```
functionName
```

*params are optional and can take positional arguments

** scope of variables

# RECITATION

**Exit codes**

Every command returns exit code: 0 = success, non-zero = error

# RECITATION

**awk**

filtering / Pattern scanning / processing

awk ‘{print $1}’ testfile.txt // print first column. default state uses white space as the separator

adding a flag to determine the separator

awk ‘{print $1}’ testfile.txt **-F,** *

# RECITATION

**sed**

altering values in text files (find/replace)

sed 's/{c_from}/{c_to}/g' <filename>

s - mode

{c_from} - word to change from

{c_to} - word to change to

g - scope.

adding the i flag makes a duplicate of the original file before it is altered

sed 's/{c_from}/{c_to}/g' <filename>

# RECITATION

**Final Mini Project**

Write a script to automatically build, compile and run your program